Guia do EDA

Homework · Previsão de recompra a 90 dias (Everything & Then Some)

Explicação, passo a passo, do notebook `Homework_EDA.ipynb`: o que cada passo faz, o que encontrou nos dados e porque é que isso interessa para o resto do trabalho.

Como ler este guia. Os números dos passos são os mesmos do notebook (Passo 1 a 42). Cada passo tem três partes: **O que faz** (o código, em palavras), **O que deu** (os resultados) e **Porque importa** (a consequência para o pré-processamento, a seleção de features ou a avaliação). No fim há um glossário dos termos estatísticos e a tabela de decisões que sai do EDA.

Uma regra que atravessa todo o notebook: o EDA **não altera os dados**. Sempre que é preciso medir sem as linhas vazias ou sem os valores impossíveis, o notebook cria uma cópia temporária (`filled`, `view`, `log_view`). O `train` original fica intacto, com as 6,534 linhas. As limpezas são decididas aqui e aplicadas no pré-processamento.

Só técnicas dadas em aula. Tudo o que o notebook usa aparece nas práticas: `describe`/skew, `log1p`, regra do IQR com boxplot, correlação de Pearson, point-biserial, qui-quadrado com V de Cramér, histogramas, boxplots, gráficos de barras e heatmaps.

Os dados em 30 segundos

ficheiro	linhas	o que é
`train.csv`	6,534	ocasiões de compra com o target `repeat_purchase_90d`
`test.csv`	986	compras posteriores, sem target; é o que se prevê no Kaggle
`clientes.csv`	1,143	atributos de cada cliente (geografia, ano de início), opcional
`sample_submission.csv`	986	o formato da submissão (`ID`, previsão 0/1)

Cada linha é uma compra, não um cliente. O target vale 1 se o mesmo cliente voltar a comprar nos 90 dias seguintes a essa compra. Um cliente que compra muitas vezes aparece em muitas linhas.

Os grandes achados, em resumo:

- o target está quase equilibrado (55.9% de recompras);
- o teste é o **futuro** do treino e quase todos os seus clientes já aparecem no treino;
- há 65 linhas vazias, missing values com explicação lógica e códigos administrativos (`-4`, sinais trocados, `1e9`);
- a frequência e a recência das compras são o sinal mais forte; o valor da compra atual quase não importa;
- `random_noise` e `document_series` não têm informação útil;
- do `clientes.csv` só três colunas geográficas valem a pena; o resto é vazio, identifica o cliente ou é redundante.

Secção 1 · Imports e dados

A célula de imports. Todos os imports estão numa só célula no topo, como nas práticas (regra PEP 8). Para que serve cada um:

import	para quê
`sys`, `pathlib.Path`	encontrar o `course_helpers.py` e construir os caminhos dos ficheiros
`warnings`	ainda não usado; serve para silenciar avisos conhecidos na modelação
`numpy`	contas numéricas: `log1p`, `nan`, máscaras dos heatmaps
`pandas`	ler os CSV, tabelas, `groupby`, `crosstab`, merge, correlações
`matplotlib`, `seaborn`	todos os gráficos
`chi2_contingency`	teste do qui-quadrado; base do V de Cramér
`pointbiserialr`	correlação entre uma numérica e o target binário
`PLOT_BLUE`, `PLOT_ORANGE`	as duas cores das práticas (do `course_helpers.py`)

Também define constantes: `RANDOM_STATE = 42` (reprodutibilidade), `FIGSIZE`, a pasta `DATA` e o nome do target, `TARGET`.

Passo 1 · Ler os quatro ficheiros

O que faz

Lê os quatro CSV. O `ID` vai para o **índice** em vez de ficar como coluna: assim nunca entra num modelo por engano, e continua disponível para a submissão. `float_precision="round_trip"` lê os valores monetários exatamente como estão escritos.

O que deu

Treino 6,534 × 38, teste 986 × 37 (sem o target), clientes 1,143 × 13, sample_submission 986 × 2.

Parte A · EDA antes do merge

Secção 2 · Levantamento estrutural

Antes de qualquer estatística: o que é cada coluna e para que serve?

Passo 2 · Dar um papel a cada coluna

O que faz

O `dtype` diz **como** o valor está guardado, não **o que mede**. Por isso as colunas são separadas à mão, pelos grupos que o enunciado descreve: target, identificador (`customer_id`), data (`purchase_date`), monetário, detalhe da compra, timing histórico, valor histórico, frequência e gasto móvel, ruído (`random_noise`) e as 6 categóricas.

No fim verifica que nenhuma coluna ficou sem papel e que o teste tem as mesmas colunas que o treino, menos o target.

O que deu

29 numéricas e 6 categóricas; as duas verificações passam.

Porque importa

`customer_id` é um número, e um `select_dtypes("number")` metia-o no modelo. O modelo aprenderia "o cliente 519184 recompra", ou seja, decoraria clientes em vez de generalizar. Ao marcá-lo como identificador, nunca entra como feature.

Passo 3 · Tabela coluna a coluna

O que faz

Para cada coluna mostra o papel, o `dtype`, quantos valores distintos tem, os missing values no treino e no teste, e se uma coluna `float` só contém números inteiros.

O que deu

Nove colunas `float` são contagens (`prior_purchase_count`, `purchase_count_*`, os dias...). São decimais só porque têm missing values: o tipo inteiro do pandas não aceita `NaN`.

Quase todas as colunas têm **exatamente 65 missing values**, um primeiro sinal das 65 linhas vazias da secção 5.

Passo 4 · Os níveis das categóricas

O que faz

Para cada categórica, quantos níveis tem e quanto pesa o nível mais frequente.

O que deu

`warehouse` (2 níveis, 95.7% num só) e `commercial_zone` (5 níveis, 95.8%) são quase constantes; `transport_method` tem 80 níveis.

Um one-hot direto daria cerca de **108 colunas** só das categóricas.

Porque importa

Uma coluna quase constante tem pouca informação, e uma com 80 níveis cria dezenas de colunas quase vazias. O pré-processamento vai ter de agrupar os níveis raros.

Secção 3 · Variável-alvo

Passo 5 · Olhar para o target

O que faz

Conta as linhas de cada classe, calcula a accuracy de um "modelo" que responde sempre a classe maioritária, e verifica que o `sample_submission` tem os mesmos `ID` que o teste, pela mesma ordem.

O que deu

3,654 recompras (55.92%) contra 2,880. Prever sempre "recompra" já dá 0.5592 de accuracy, sem acertar nenhum dos 2,880 negativos.

O `sample_submission` prevê 1 em 75% das linhas. É só uma baseline, não a resposta.

Porque importa

Como a accuracy de 0.5592 se obtém sem pensar, a avaliação deve usar métricas que olham para as duas classes (F1 e AUC-ROC, da semana 5). 0.5592 é o valor mínimo a bater.

Secção 4 · Duplicados, clientes repetidos e tempo

Passo 6 · Contar duplicados

O que faz

`train.duplicated()` marca as linhas iguais a uma anterior (o `ID` está no índice, por isso não entra na comparação). Com `keep=False` marca **todas** as cópias, incluindo a primeira.

Envolvidas vs excesso. Se três linhas são iguais, há 3 envolvidas e 2 em excesso (as que se podiam apagar). Excesso = envolvidas − número de grupos.

O que deu

10 linhas em excesso, 16 envolvidas, ou seja, 6 grupos de linhas iguais. **As 16 são todas linhas vazias**: só têm `customer_id` e target, por isso ficam iguais entre si.

Nas linhas com dados não há nenhum duplicado, nem duas compras do mesmo cliente na mesma data.

Porque importa

Os dados reais não têm duplicados. Os 10 desaparecem quando se removerem as linhas vazias.

Passo 7 · Clientes repetidos

O que faz

`train.groupby("customer_id").size()` conta quantas compras tem cada cliente. Repetir o cliente é normal: cada linha é uma compra.

Depois compara os clientes do treino com os do teste (`set(...) & set(...)` dá os que estão nos dois).

O que deu

610 clientes para 6,534 linhas. Mediana de 4 compras, média 10.7, máximo 277. Um quarto dos clientes tem uma só compra. Os 10 mais ativos concentram 17.5% do treino.

184 dos 206 clientes do teste já estão no treino, e cobrem 932 das 986 linhas.

Porque importa

Amostra efetiva de 610. Uma variável do cliente (região, ilha...) repete-se em todas as compras desse cliente. Se a região RG24 aparece em 100 linhas, mas todas do mesmo cliente, só sabemos uma coisa: esse cliente, que é de RG24, recompra muito. Para variáveis do cliente há 610 observações independentes, não 6,534.

Daí vêm três cuidados: desconfiar de taxas por categoria com poucos **clientes**, agrupar os níveis raros e evitar colunas que identificam o cliente.

Passo 8 · Treino e teste no tempo

O que faz

Converte `purchase_date` em datas (`to_datetime`), compara a primeira e a última data de cada ficheiro, e conta as compras e a taxa de recompra por ano.

O que deu

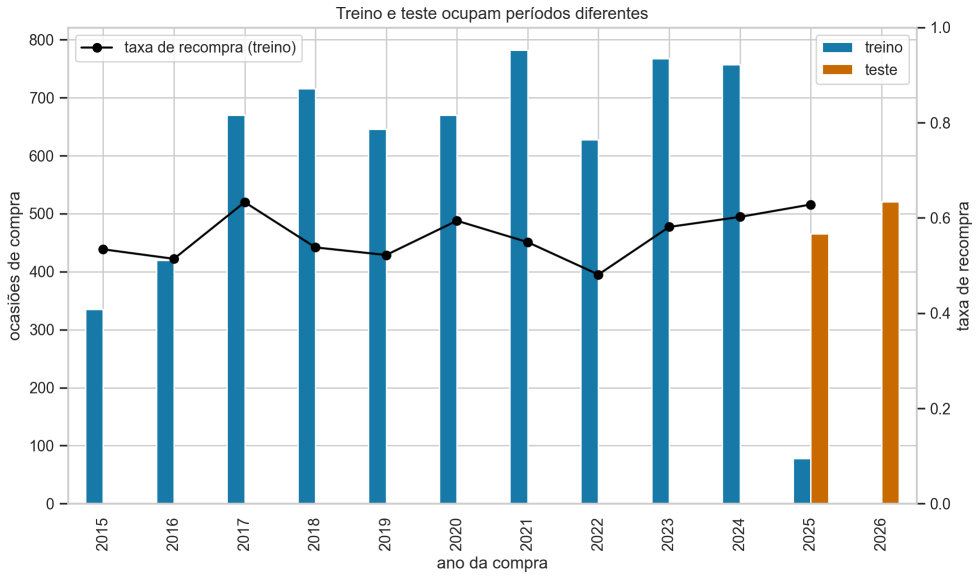
Treino: 2015-07-11 a 2025-02-15. Teste: 2025-05-24 a 2026-10-06. **98 dias de intervalo.** O teste é o **futuro** do treino.

O intervalo provavelmente existe porque o target olha 90 dias para a frente (interpretação nossa, não está no enunciado).

A taxa de recompra anual oscila entre 0.48 (2022) e 0.63 (2017 e 2025), sem tendência.

Porque importa

Se a validação baralhar as linhas ao acaso, o modelo treina com compras de 2024 e é testado com compras de 2018, ou seja, "vê o futuro" e o score sai otimista. É melhor validar respeitando o tempo (treinar no passado, validar no mais recente), por exemplo com o `TimeSeriesSplit` da semana 5.



Passo 8: compras por ano no treino (azul) e no teste (laranja), e taxa de recompra por ano (linha preta).

Secção 5 · Missing values

Passo 9 · Visão geral e linhas vazias

O que faz

Conta as linhas e colunas com missing values, e identifica as linhas em que **nenhuma** feature está preenchida (`empty_rows`).

O que deu

2,605 linhas (39.9%) têm pelo menos um missing value; 36 das 38 colunas têm algum.

65 linhas vazias no treino, 0 no teste. Só têm `customer_id` e target, e a taxa de recompra delas (0.508) não difere muito da global.

Porque importa

Não há nada para imputar a partir destas linhas e o teste não tem nenhuma. São candidatas naturais a remoção no pré-processamento.

Passo 10 · Os missing values que sobram

O que faz

`filled = train[~empty_rows]` cria uma **cópia** sem as 65 linhas; **não apaga nada** do `train`. Serve para que as 65 linhas não escondam o padrão das outras.

Depois mostra os missing values de cada coluna no treino e no teste.

O que deu

As colunas monetárias desaparecem da tabela: os seus 65 missing values eram todos das linhas vazias. Ficam dois grupos:

• **histórico** (`prior_*_gap_days`, `prior_*_purchase_value`, `days_since_previous_purchase`): 9% a 22%;

• **cinco colunas com ~3% cada**: `payment_type`, `salesperson`, `transport_method`, `warehouse`, `spend_365d`.

No teste, os missing values do histórico são menores (2% a 8%).

Passo 11 · Missing values estruturais

O que faz

Hipótese: os missing values do histórico são lógicos. Sem compra anterior não há "dias desde a compra anterior"; um intervalo precisa de 2 compras; um desvio-padrão dos intervalos precisa de 3.

`pd.cut` divide as linhas por `prior_purchase_count` (0, 1, 2, 3+) e o heatmap mostra a % de missing values de cada coluna em cada grupo.

O que deu

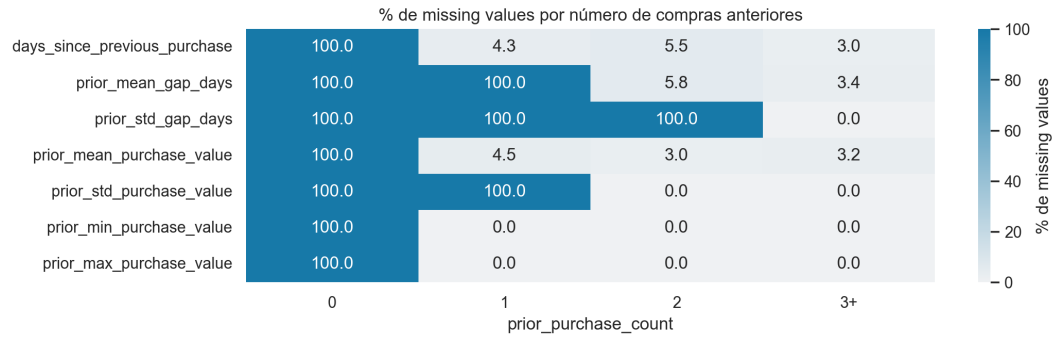
Os 100% formam uma **escada**, exatamente onde a lógica diz que a coluna não pode existir: com 0 compras falta tudo; com 1 faltam os intervalos e o desvio-padrão dos valores; com 2 só o desvio-padrão dos intervalos; com 3+ quase nada.

Sobram 3-6% de missing values sem explicação em clientes com histórico.

Porque importa

Estes missing values são **"não se aplica"**, não "desconhecido". Imputá-los pela mediana dá a um cliente na primeira compra `days_since_previous_purchase = 57`, um histórico inventado.

Melhor: um indicador de primeira compra e um preenchimento com significado (por exemplo, 0 de desvio-padrão quando só há uma compra). Os 3-6% aleatórios imputam-se normalmente.



Passo 11: % de missing values por número de compras anteriores.

Passo 12 · Os missing values de ~3%

O que faz

Verifica se as cinco colunas têm missing values **nas mesmas linhas** (matriz de co-ocorrência) e se a flag "tem missing value" está ligada a alguma categórica observada, com qui-quadrado e V de Cramér (como na semana 3).

O target fica de fora, de propósito: se fosse o label a decidir a limpeza, seria uma fuga de informação.

O que deu

Nenhuma linha tem missing values em mais de 2 das 5 colunas, ou seja, não aparecem em bloco.

A flag não está associada a `commercial_zone` (V 0.035) nem a `document_series` (V 0.021), com $p > 0.05$, e as medianas do histórico são parecidas com e sem missing value.

Porque importa

São compatíveis com missing values completamente aleatórios (MCAR). Uma imputação simples (moda nas categóricas, mediana em `spend_365d`, ajustada só no treino) é defensável.

Secção 6 · Valores impossíveis e regras de domínio

A regra, escrita antes de medir: dias, contagens e valores gastos não podem ser negativos. Um valor fora do domínio não é um valor pequeno, não é um valor. `random_noise` é a exceção: é uma normal centrada em zero, por isso os negativos são legítimos.

Passo 13 · Encontrar os negativos

O que faz

Conta os negativos em todas as numéricas exceto `random_noise`, e mostra quantos valores distintos têm e o mais frequente.

O que deu

`days_since_previous_purchase`: 16 negativos, **todos -4**. `prior_mean_gap_days`: 16, **todos -4**.

`prior_mean_purchase_value`: 16 negativos, todos diferentes. `spend_365d`: 15, todos diferentes.

O teste tem os mesmos problemas (2 em cada coluna).

Passo 14 · Código ou dano aleatório?

O que faz

Valores espalhados indicam dano aleatório; um único valor repetido indica um **código** de preenchimento.

Para as médias negativas, compara com `prior_total_spend / prior_purchase_count`, que é o que a média devia ser. Para `spend_365d`, compara com `spend_180d`.

O que deu

Os `-4` estão em 32 linhas diferentes (16 numa coluna, 16 noutra, nenhuma com as duas): é um código, e nenhum dia é negativo.

As 16 médias negativas são exatamente −(total / contagem): o valor certo com o sinal trocado.

Nos 15 `spend_365d` negativos, o valor absoluto é sempre ≥ `spend_180d`, consistente com um sinal trocado.

Porque importa

Os sinais trocados não precisam de imputação: o valor verdadeiro pode ser **reconstruído** pela identidade.

Passo 15 · O placeholder 10^9

O que faz

Procura o valor 1,000,000,000 em todas as numéricas.

O que deu

16 em `prior_mean_purchase_value` e 15 em `spend_365d`, mais 2 + 2 no teste. Os maiores valores legítimos são 72,937 e 522,933.

Um único 10^9 põe a média de `spend_365d` em 2.4 milhões; por isso o `describe()` original parecia absurdo.

Porque importa

Como o teste tem os mesmos códigos, a correção tem de ser uma função aplicada da mesma forma aos dois ficheiros.

Passo 16 · A vista temporária `view`

O que faz

Cria `view`: as linhas com dados, com os `-4`, os negativos e os 10^9 passados a `NaN`. **Não é a limpeza final**, é só para que um código não decida uma média ou uma correlação.

O que deu

32 valores impossíveis em `prior_mean_purchase_value`, 30 em `spend_365d`, 16 em cada uma das colunas de dias; no teste, 2 a 4 por coluna.

Secção 7 · Consistência entre colunas

Passo 17 · Identidades contabilísticas

O que faz

Muitas colunas derivam umas das outras. Cada identidade (soma, ordem, divisão) é uma verificação grátis da qualidade dos dados. Conta as linhas que a violam, com 1 cêntimo de tolerância.

O que deu

`Total Bruto = Liquido + Desc_ Linha + Desc_ Global` e `Total a Pagar = Liquido + Imp_`: **0 violações**. Os portes já estão dentro do líquido (somá-los outra vez falha nas 1,053 compras com portes).

As janelas móveis são sempre monótonas (30d ≤ 90d ≤ 180d ≤ 365d); quem não tem compras anteriores tem sempre gasto anterior 0.

`Custo = 0` em 486 linhas e custo acima do líquido em 39: são plausíveis (ofertas, vendas com prejuízo), ficam como estão.

Porque importa

As colunas ligadas por identidades exatas são **redundantes por construção**: um modelo não ganha nada com as três medidas do total.

Passo 18 · Médias inflacionadas

O que faz

Com os códigos já tapados, mede quanto as médias que ainda não batem certo se afastam de total / contagem.

O que deu

46 médias estão inflacionadas entre 8.5× e 24.8×, e 45 delas ficam fora de [mínimo, máximo], o que é impossível (a média não pode passar o máximo).

Porque importa

Tal como os sinais trocados, reconstroem-se pela identidade, que pode ser aplicada a toda a coluna.

Secção 8 · Distribuições e outliers

Passo 19 · Estatísticas de forma

O que faz

`describe()` mais o que não reporta: a razão média/mediana, a assimetria (skew) e a % de zeros (como na semana 3).

O que deu

25 das 29 numéricas têm skew acima de 3. Nos valores da compra e no histórico, a média é 1.9 a 2.9 vezes a mediana (`Total a Pagar` 926 contra 477).

Algumas colunas são quase só zeros: `Total Desc_ Global` 98.3%, `Total Portes` 83.7%, `spend_30d` 67%.

Passo 20 · Histogramas em log1p

O que faz

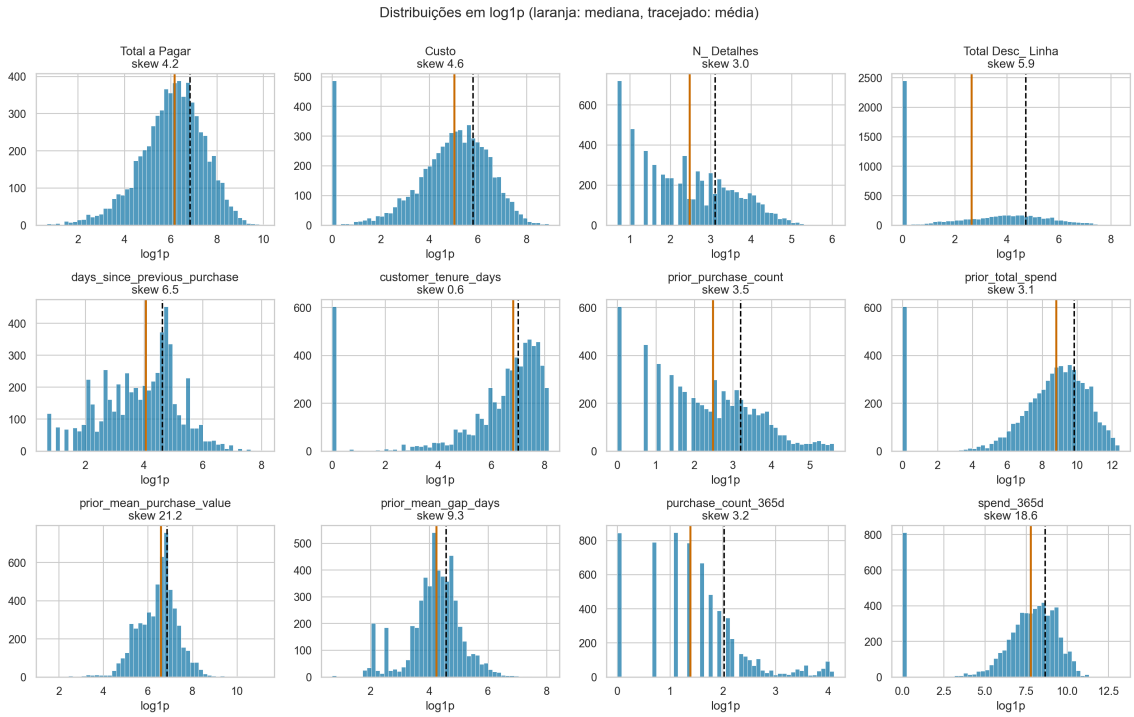
Desenha as colunas principais em escala `log1p` (log(1 + x)), porque na escala original a cauda esconde tudo. A linha laranja é a mediana e a tracejada a média.

O que deu

Em `log1p` as distribuições ficam aproximadamente simétricas.

Porque importa

Sugere aplicar `log1p` antes de modelos sensíveis à escala, como a regressão logística e o KNN.



Passo 20: distribuições em log1p (laranja: mediana; tracejado: média).

Passo 21 · Onde a regra do boxplot falha

O que faz

A regra do boxplot marca como outlier o que estiver a mais de 1.5 × IQR dos quartis (IQR = Q3 − Q1). Conta quantas linhas assinala em cada coluna, ao lado da % de zeros.

Compara `Total a Pagar` (uma coluna com "meio") com `Total Portes` (quase só zeros).

O que deu

Em `Total Portes`, Q1 = Q3 = 0, logo IQR = 0, e a regra assinala **todas** as 1,053 compras com portes: mede quantos valores não são zero, não quantos são anómalos.

Nas colunas com "meio", assinala 5-10% das linhas, que são clientes grandes, não erros (os erros já foram tratados na secção 6).

Porque importa

Apagar estes "outliers" tiraria do treino exatamente os clientes mais ativos. As alternativas (manter, `log1p`, limitar os extremos) comparam-se no pré-processamento.

Secção 9 · Variáveis categóricas

Passo 22 · Níveis e taxa de recompra

O que faz

Para cada categórica: níveis, níveis raros (menos de 1% das linhas), % de missing values e um gráfico com a taxa de recompra por nível. Cada barra mostra o número de linhas, porque uma taxa com poucas linhas é instável.

O que deu

`warehouse` (99.7% num nível) e `commercial_zone` (96.8%) são quase constantes.

`payment_type` e `salesperson` têm diferenças visíveis entre os níveis frequentes e 4-5 níveis raros cada.

Porque importa

Os níveis raros devem ser agrupados num nível "outros" antes de codificar.

Passo 23 · `transport_method`

O que faz

Com 80 níveis não cabe num gráfico de barras. Mostra quantos níveis são raros e, para os 10 mais frequentes, as linhas, a taxa de recompra e **o número de clientes**.

O que deu

59 dos 80 níveis têm menos de 1% das linhas, e 40 têm menos de 10.

O nível 11 tem 253 linhas, 98% de recompra e só 2 clientes.

Porque importa

Um nível destes não descreve um método de transporte, identifica um cliente. O modelo decoraria esses 2 clientes. É preciso agrupar os níveis raros ou usar uma codificação por frequência.

Passo 24 · `document_series` mede o tempo

O que faz

Cruza a série do documento com o ano da compra.

O que deu

A série 3 vai de 2015 a 2021, a 1 de 2017 a 2021 e a 2 começa em 2021. **O teste é 100% série 2.**

A taxa de recompra é igual nas três séries (V = 0.003).

Porque importa

No teste a coluna é constante, logo não tem informação. Deve ser removida.

Secção 10 · Relação com o target

Passo 25 · Tabela de relevância

O que faz

Uma medida por coluna, escolhida pelo tipo (semana 4): **|r| point-biserial** para as numéricas e **V de Cramér** para as categóricas. As duas vão de 0 a 1.

Para as numéricas, o r é calculado duas vezes: na escala original e em `log1p`, porque a cauda distorce o r. O gráfico mostra as barras em `log1p` e um traço preto na escala original.

O que deu

A frequência recente lidera: `purchase_count_365d`, `_180d` e `_90d` com |r| 0.40-0.42 em log1p, seguidas do gasto móvel e de `prior_purchase_count`.

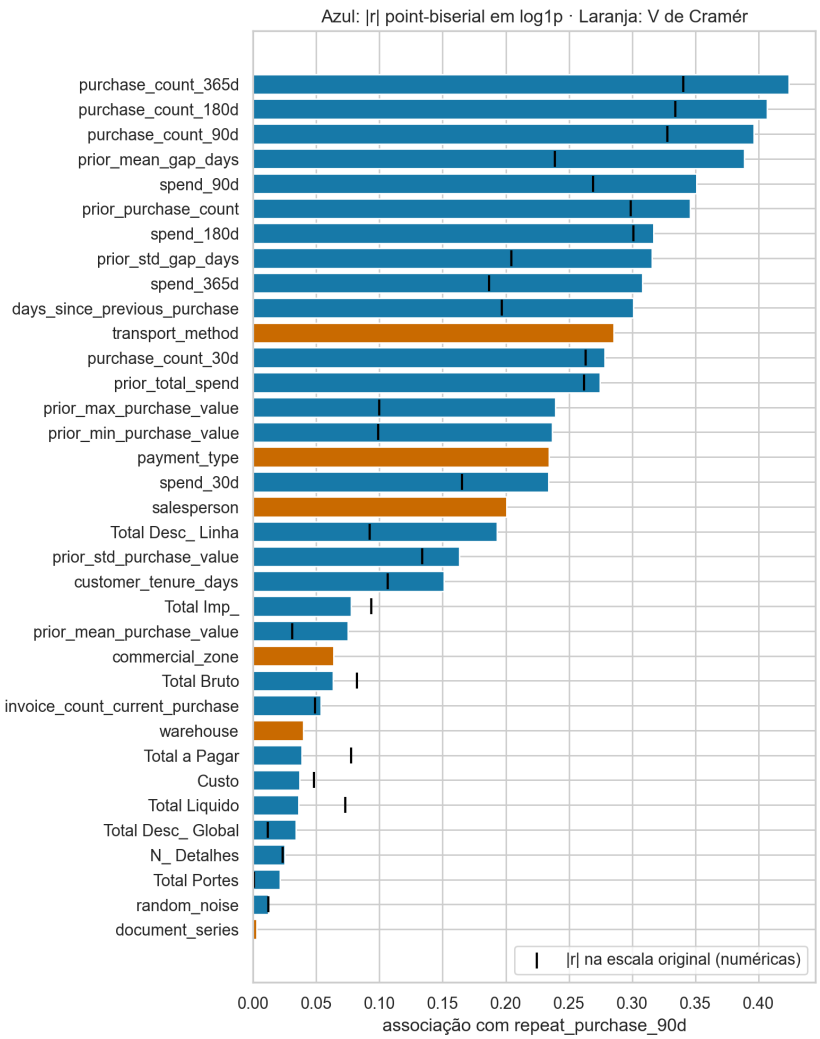
O `log1p` muda a leitura: `prior_mean_gap_days` passa de 0.24 para 0.39, `prior_max_purchase_value` de 0.10 para 0.24.

Nas categóricas, `transport_method` (V 0.29) e `payment_type` (0.23), mas o V de `transport_method` está inflacionado pelos níveis que identificam clientes.

O valor da compra atual quase não importa (|r| abaixo de 0.08).

Porque importa

Diz que features são candidatas fortes e é um argumento direto para usar `log1p`.



Passo 25: associação de cada coluna com o target (azul: |r| em log1p; laranja: V de Cramér; traço: |r| na escala original).

Passo 26 · Boxplots por classe

O que faz

Para as seis numéricas mais associadas, um boxplot em `log1p` para cada classe do target.

O que deu

As caixas da classe 1 (recompra) estão claramente deslocadas: mais compras recentes, intervalos mais curtos.

Passo 27 · Recência e frequência

O que faz

Taxa de recompra por decil de `days_since_previous_purchase` (`pd.qcut` divide em 10 grupos do mesmo tamanho) e por número de compras no último ano.

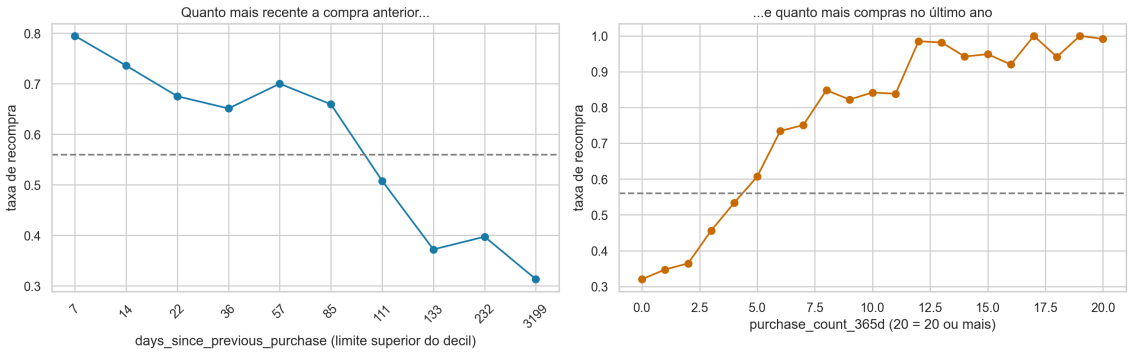
O que deu

A taxa desce de **0.80** no decil mais recente para **0.31** no mais antigo.

Sobe de **0.32** sem compras no último ano para perto de **1** a partir de 12.

Porque importa

É o padrão RFM clássico (recência, frequência, valor): quem compra muito e há pouco tempo volta a comprar.



Passo 27: taxa de recompra por recência (esquerda) e frequência (direita).

Passo 28 · `random_noise`

O que faz

O enunciado pede para avaliar se esta coluna tem informação. Descreve-a, testa a associação com o target e compara as densidades nas duas classes.

O que deu

Normal padrão (média −0.02, desvio-padrão 1.00, skew 0.05), r = −0.012 com p = 0.335, e nenhuma correlação acima de 0.03 com outra feature. As duas classes têm a mesma densidade.

Porque importa

É ruído. Fica como candidata a remoção, a confirmar na seleção de features.

Secção 11 · Relações multivariadas e redundância

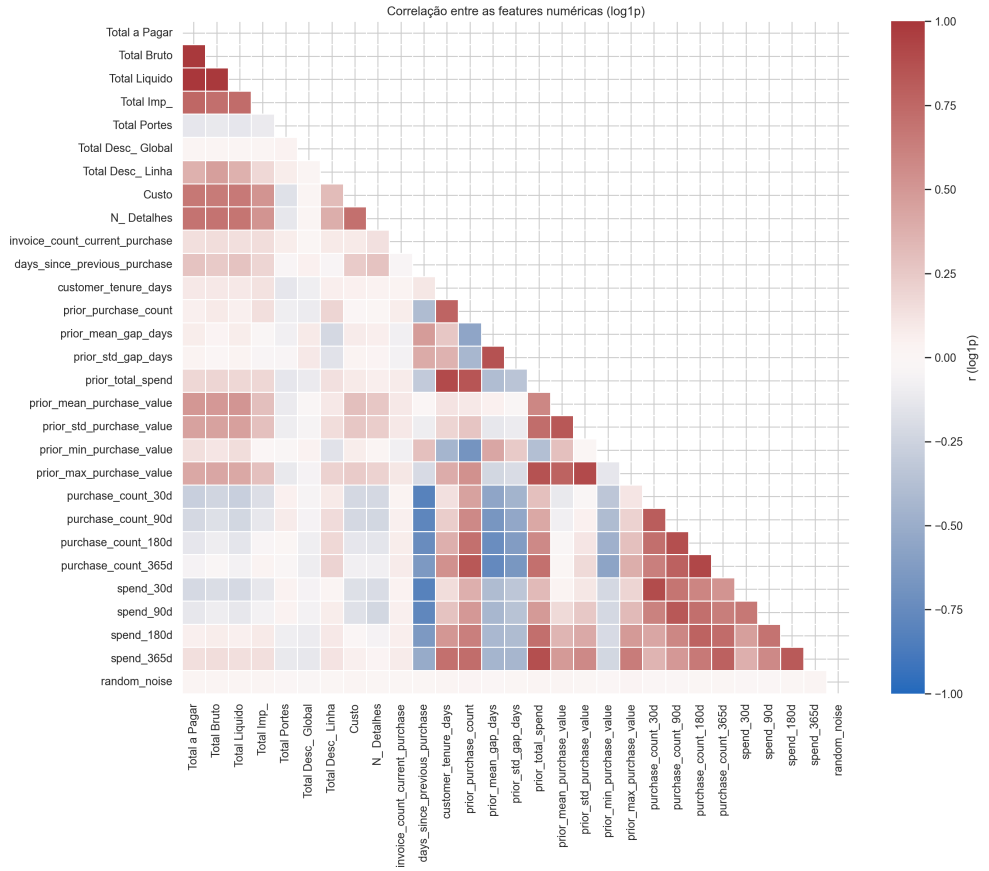
Passo 29 · Heatmap de correlação

O que faz

Correlação de Pearson entre todas as numéricas, em `log1p`.

O que deu

Vêem-se **blocos**: as janelas móveis correlacionam-se fortemente entre si, e o bloco monetário da compra atual quase não se correlaciona com o histórico.



Passo 29: correlação (r em log1p) entre as features numéricas.

Passo 30 · Pares redundantes

O que faz

Lista os pares com |r| ≥ 0.9.

O que deu

6 pares: `Total a Pagar`, `Total Bruto` e `Total Liquido` entre si (0.99-1.00), `purchase_count_180d` com `_365d` (0.92), `customer_tenure_days` com `prior_total_spend` (0.90), e `prior_std` com `prior_max_purchase_value` (0.90).

Porque importa

Na seleção de features, deve manter-se um representante por bloco (o mais associado ao target), em vez de todos.

Passo 31 · Categóricas entre si

O que faz

V de Cramér entre cada par de categóricas.

O que deu

Nenhum par se aproxima de 1: nenhuma categórica é outra com outro nome.

Secção 12 · Treino vs teste

Olhar para o teste **para descrever** não usa informação dele para ajustar nada; o que não se pode é ajustar transformações com ele.

Passo 32 · Medianas treino vs teste

O que faz

Compara a mediana de cada numérica nos dois ficheiros e a razão teste/treino.

O que deu

Os clientes do teste são **mais antigos e mais ativos**: `customer_tenure_days` 2.9× (2,662 contra 918 dias), `prior_purchase_count` 2.6× (28 contra 11), `prior_total_spend` 3.7×.

Passo 33 · Categóricas treino vs teste

O que faz

O peso do nível dominante em cada lado, e os níveis do teste que nunca aparecem no treino.

O que deu

`document_series`: 100% série 2 no teste.

`commercial_zone` passa de 96.8% para 70.1% na zona dominante, e **209 linhas do teste (21%) estão em zonas que o treino nunca viu**; `transport_method` tem 63.

Porque importa

O one-hot ajustado no treino tem de aceitar níveis desconhecidos (`handle_unknown="ignore"`).

Passo 34 · Clientes que voltam no teste

O que faz

Compara, no treino, os 184 clientes que voltam a aparecer no teste com os restantes.

O que deu

Os que voltam recompram a **0.618** (mediana de 4 compras no último ano); os outros 423 a **0.467** (mediana de 2).

Porque importa

O teste é feito sobretudo de clientes fiéis, o que ajuda a perceber porque a baseline do `sample_submission` prevê 1 em 75% das linhas. Reforça a validação temporal.

Secção 13 · Resumo antes do merge

#	achado	decisão para o pré-processamento
1	65 linhas vazias (10 duplicadas); o teste não tem nenhuma	remover do treino
2	missing values estruturais no histórico	indicador de primeira compra + preenchimento com significado
3	~195 missing values aleatórios em 4 categóricas e `spend_365d`	moda / mediana ajustadas só no treino
4	`-4`, sinais trocados, 10^9, médias inflacionadas	`-4` e 10^9 → missing value; sinais e médias → reconstruir pelas identidades
5	identidades exatas; 6 pares com $\|r\| \geq 0.9$	um representante por bloco na seleção
6	assimetria forte, picos em zero, IQR inútil	`log1p` + escalonamento; outliers são clientes grandes
7	`transport_method` com 80 níveis; níveis raros	agrupar níveis raros; one-hot tolerante a níveis novos
8	`document_series` mede o tempo; `warehouse` quase constante	remover
9	`random_noise` sem associação	candidata a remoção
10	teste posterior, clientes mais antigos, zonas novas	validação temporal

Parte B · EDA depois do merge

O enunciado deixa a junção como opcional ("if useful"). A pergunta desta parte: o que é que o `clientes.csv` acrescenta, e a que custo?

Secção 14 · Levantamento de clientes.csv

Passo 35 · Estrutura da tabela de clientes

O que faz

O mesmo levantamento da secção 2: tipo, valores distintos, % de distintos e missing values de cada coluna.

O que deu

`customer_id` é único (um cliente por linha).

`customer_type` **está 100% vazia**; `company_name` (1,141 valores) e `address` (1,120) são praticamente um por cliente: identificadores.

`island` e `street` têm mais de 50% de missing values.

Passo 36 · Cobertura da chave

O que faz

Verifica se todos os clientes do treino e do teste estão no `clientes.csv`.

O que deu

Todos estão. Há ainda 511 clientes sem compras nestes ficheiros, que o left join ignora.

Passo 37 · A hierarquia geográfica

O que faz

As colunas geográficas vão do mais largo ao mais fino (país, região, ilha, município, cidade, código postal, rua, morada). Para cada uma, a mediana de clientes por nível e os níveis com um só cliente. Mostra também `island` por país e o histograma de `customer_since_year`.

O que deu

`city`, `postal_code`, `street` e `address` têm mediana de **1 cliente por nível**, ou seja, identificam o cliente. `municipality` e `postal_area` (2) estão no limite.

`island` está vazia em 100% dos clientes fora do país CT10 e em 47.6% dos de CT10: o missing value quer dizer "sem ilha" (presumivelmente o continente), não "desconhecido".

58.1% dos clientes têm `customer_since_year = 2015`, o ano em que os dados começam: "2015" quer dizer "2015 ou antes" (censura à esquerda).

Porque importa

Só `country_or_region`, `commercial_region` e `island` têm níveis com clientes suficientes para serem categorias reais.

Secção 15 · Merge por customer_id

Passo 38 · O merge

O que faz

Left join de cada compra ao seu cliente. `validate="many_to_one"` faz o pandas falhar se um `customer_id` estiver repetido no `clientes.csv`, o que duplicaria compras em silêncio. O mesmo merge é aplicado ao teste (só para o descrever), e o índice `ID` é preservado.

O que deu

Treino 6,534 × 38 → 6,534 × 50; teste 986 × 37 → 986 × 49. Mesmas linhas, mesma ordem, nenhuma compra sem cliente.

Porque importa

A função `merge_clients` é a que o pré-processamento deve usar no treino e no teste, para os dois passarem exatamente pelo mesmo passo.

Secção 16 · Qualidade das novas colunas

Passo 39 · Missing values por cliente e por linha

O que faz

Compara a % de missing values contada por cliente com a contada por linha (compra).

O que deu

`island` tem missing values em 46.9% dos clientes mas só em 33.8% das linhas: os clientes com ilha preenchida compram mais vezes.

Porque importa

Qualquer estatística destas colunas pesa os clientes pelo número de compras.

Passo 40 · Consistência temporal

O que faz

Verifica que o cliente não começou depois da compra e compara `customer_since_year` com a antiguidade em `customer_tenure_days`.

O que deu

Nenhuma violação.

Convertida em anos de relação (ano da compra − `customer_since_year`), tem correlação **0.92** com `customer_tenure_days`.

Porque importa

`customer_since_year` é quase redundante; o pouco que acrescenta vem da censura de 2015.

Secção 17 · Novas colunas e o target

Passo 41 · Relevância das colunas de cliente

O que faz

A mesma tabela da secção 10, com o número de **clientes por nível** ao lado do V de Cramér.

O que deu

Os V mais altos são de `company_name` (0.59), `address` (0.59), `postal_code` (0.57) e `city` (0.43), todas com 1 cliente por nível. **Não significam nada**: dizem só que a recompra varia entre clientes.

Com níveis largos: `commercial_region` (V 0.26, mediana de 9 clientes por nível), `island` (0.15), `customer_since_year` (|r| 0.12) e `country_or_region` (0.07).

Porque importa

As colunas que identificam o cliente levariam o modelo a decorar clientes, ainda mais porque quase todos os do teste estão no treino.

Passo 42 · Taxa de recompra por região, ilha e ano de início

O que faz

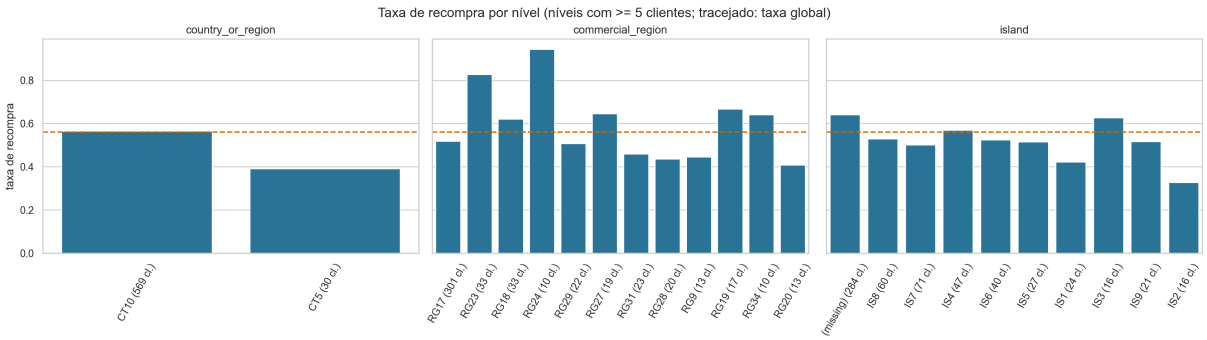
Gráficos da taxa de recompra por nível, só com níveis de pelo menos 5 clientes (o número de clientes aparece no rótulo), e por `customer_since_year`.

O que deu

RG24 (0.95) e RG23 (0.82) recompram muito acima da média (0.56), mas com 10 e 33 clientes.

Sem ilha: 0.64. País CT5: 0.39.

Clientes mais recentes recompram menos (0.25 nos de 2022, 0.60 nos de 2015), o mesmo sinal que já está na antiguidade e no número de compras.



Passo 42: taxa de recompra por país, região comercial e ilha (cl. = clientes no nível).

Secção 18 · Resumo final

Vale a pena fazer o merge? Sim, mas só de três colunas.

coluna de clientes.csv	decisão	porquê
`commercial_region`	manter, agrupando regiões raras	o sinal mais forte, níveis com clientes suficientes
`island`	manter, missing value = nível próprio	o missing value significa "sem ilha"
`country_or_region`	manter ou agrupar (CT10 / CT5 / outros)	sinal fraco mas legítimo
`customer_since_year`	comparar na seleção	quase redundante com a antiguidade (r 0.92)
`municipality`, `postal_area`	remover	1-2 clientes por nível
`city`, `postal_code`, `street`, `address`, `company_name`	remover	identificadores
`customer_type`	remover	100% missing values

Para o pré-processamento: todas as transformações (imputação, agrupamento de níveis, codificação, escalonamento) são **ajustadas só no treino** e aplicadas ao teste pela mesma função; a validação respeita a ordem temporal; o teste passa pelo mesmo `merge_clients`.

Glossário

Missing value. Uma célula sem valor (`NaN`). Pode ser desconhecida (o valor existe mas não foi registado) ou "não se aplica" (o valor não pode existir, como os dias desde a compra anterior numa primeira compra).

MCAR. Missing completely at random: o missing value não depende de nada, nem do valor escondido nem de outras colunas. Uma imputação simples só custa precisão.

Domínio. O conjunto de valores que uma coluna pode tomar, fixado pelo que mede (dias ≥ 0). Um valor fora do domínio é impossível, não extremo.

Placeholder / código. Um valor inventado para "preencher" (`-4`, `1e9`). Reconhece-se por se repetir exatamente, ou por ser absurdo.

log1p. log(1 + x). Comprime as caudas das colunas muito assimétricas e funciona com zeros (log1p(0) = 0).

Skew (assimetria). Mede quanto a distribuição tem uma cauda para um lado. Acima de 1 já é bastante assimétrica; aqui há colunas acima de 20.

IQR e regra do boxplot. IQR = Q3 − Q1, a largura dos 50% centrais. A regra marca como outlier o que estiver a mais de 1.5 × IQR dos quartis. Falha quando Q1 = Q3.

Correlação de Pearson (r). De −1 a 1: quanto duas numéricas variam juntas em linha reta. É sensível às caudas, por isso aqui é calculada em log1p.

Point-biserial. A correlação de Pearson quando uma das variáveis é binária (o target 0/1).

Qui-quadrado e V de Cramér. O qui-quadrado mede a distância entre a tabela de contagens observada e a esperada se as duas categóricas fossem independentes. O V de Cramér reescala isso para 0 a 1, para se poderem comparar colunas. O p-value só é fiável se todas as contagens esperadas forem pelo menos ~5.

Cardinalidade. O número de níveis distintos de uma categórica.

Amostra efetiva. O número de observações realmente independentes. Para variáveis do cliente são os 610 clientes, não as 6,534 compras.

Left join / validate. Junta a cada linha do treino a linha do seu cliente, mantendo todas as compras. `validate="many_to_one"` garante que cada cliente aparece uma só vez no `clientes.csv`.

Fuga de informação (leakage). Usar, para treinar ou decidir a limpeza, informação que o modelo não teria no momento de prever: o target, ou dados do teste.